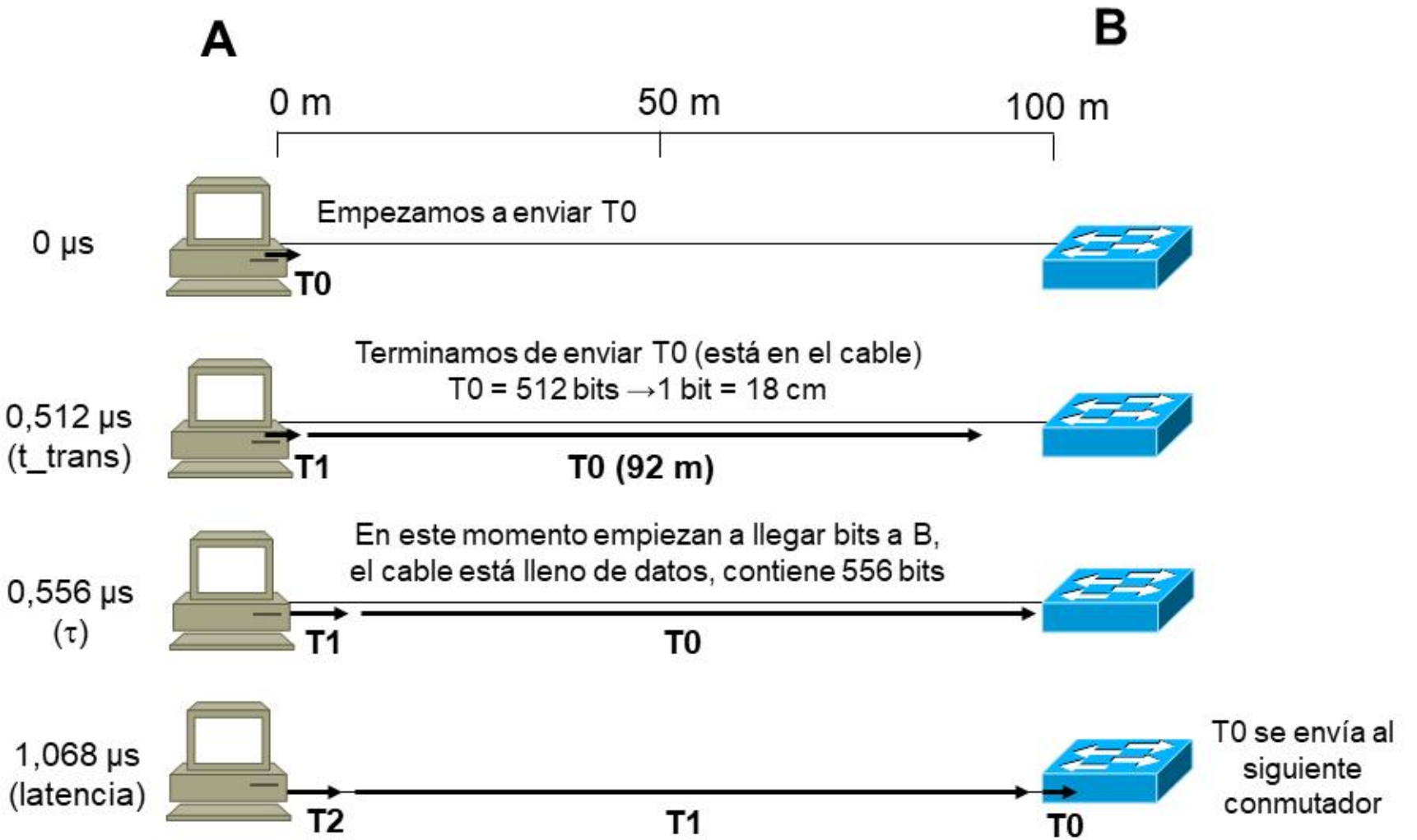
Sumario

- Funciones de la capa de enlace
- Delimitación de tramas
- Control de errores
- Definición de tiempos
- Modos de conmutación
- Protocolos de nivel de enlace
- Protocolo PPP y redes SONET/SDH

Protocolo optimista

- Tenemos un ordenador conectado a un conmutador por 1000BASE-T con un cable de 100 m, que envía continuamente tramas de 64 bytes. Supongamos que:
  - Las tramas nunca se pierden ni se producen errores.
  - El receptor tiene una capacidad de proceso infinita, nunca se desborda. No es necesario ejercer control de flujo
- En este caso no hace falta esperar ninguna confirmación. Podemos enviar tramas ininterrumpidamente

Protocolo optimista



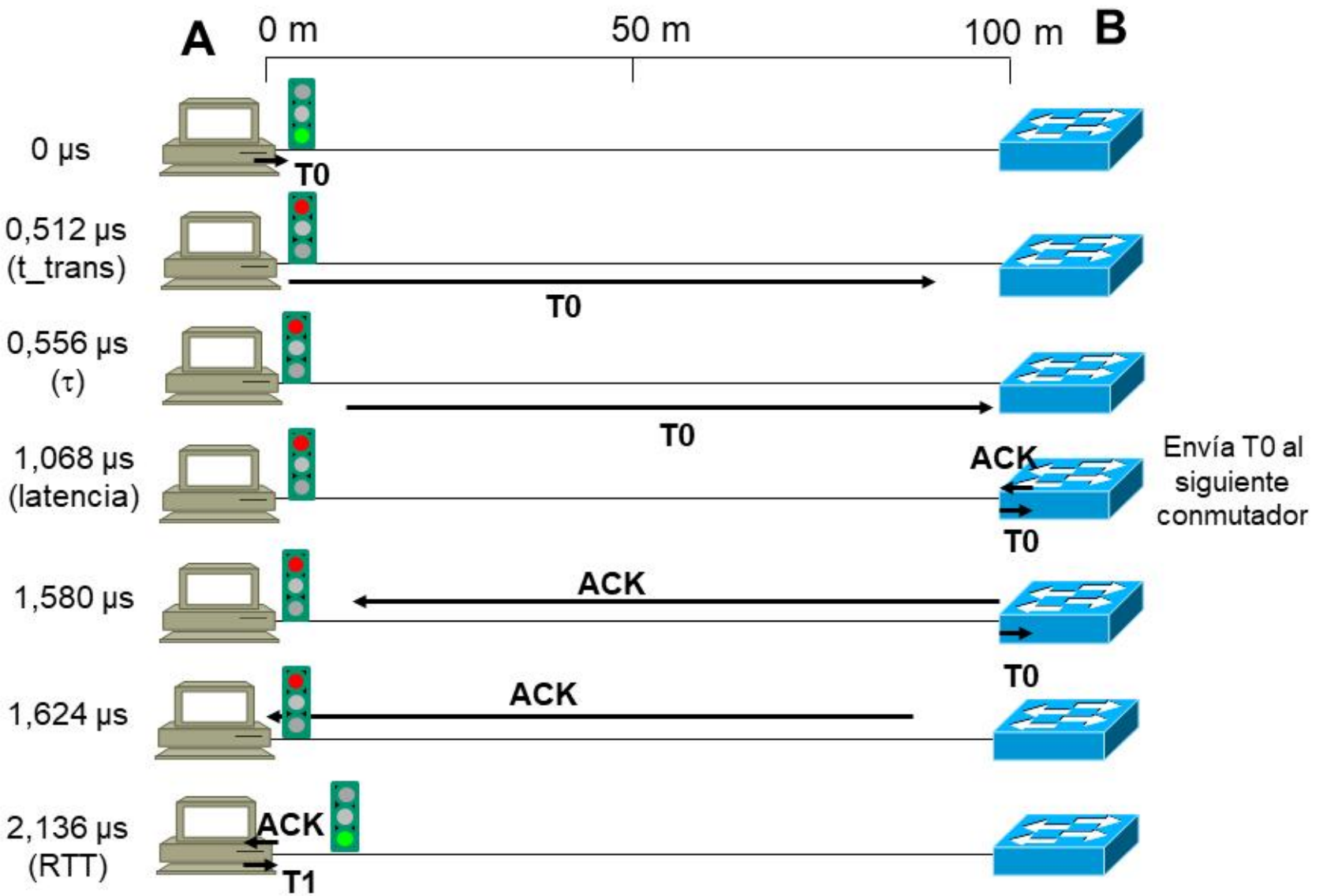
Eficiencia del protocolo optimista

- Puesto que la línea se ocupa todo el tiempo el rendimiento es del 100%. Aprovechamos el enlace en toda su capacidad

Protocolo con control de flujo

- Supongamos ahora que el nivel de red en el receptor no es capaz de digerir los datos al ritmo enviado. En ese caso el buffer se llenará y las tramas se perderán
- Para evitarlo vamos a cambiar el protocolo. Ahora cada vez que enviemos una trama esperaremos una confirmación antes de mandar la siguiente
- La confirmación, o acuse de recibo, es lo que llamamos un ACK (de 'acknowledgement', acuse de recibo en inglés). Si recibimos el ACK enviamos la siguiente trama, si no esperamos. Es como si tuviéramos un semáforo que cuando enviamos una trama se pone en rojo y que al recibir el ACK se pone en verde

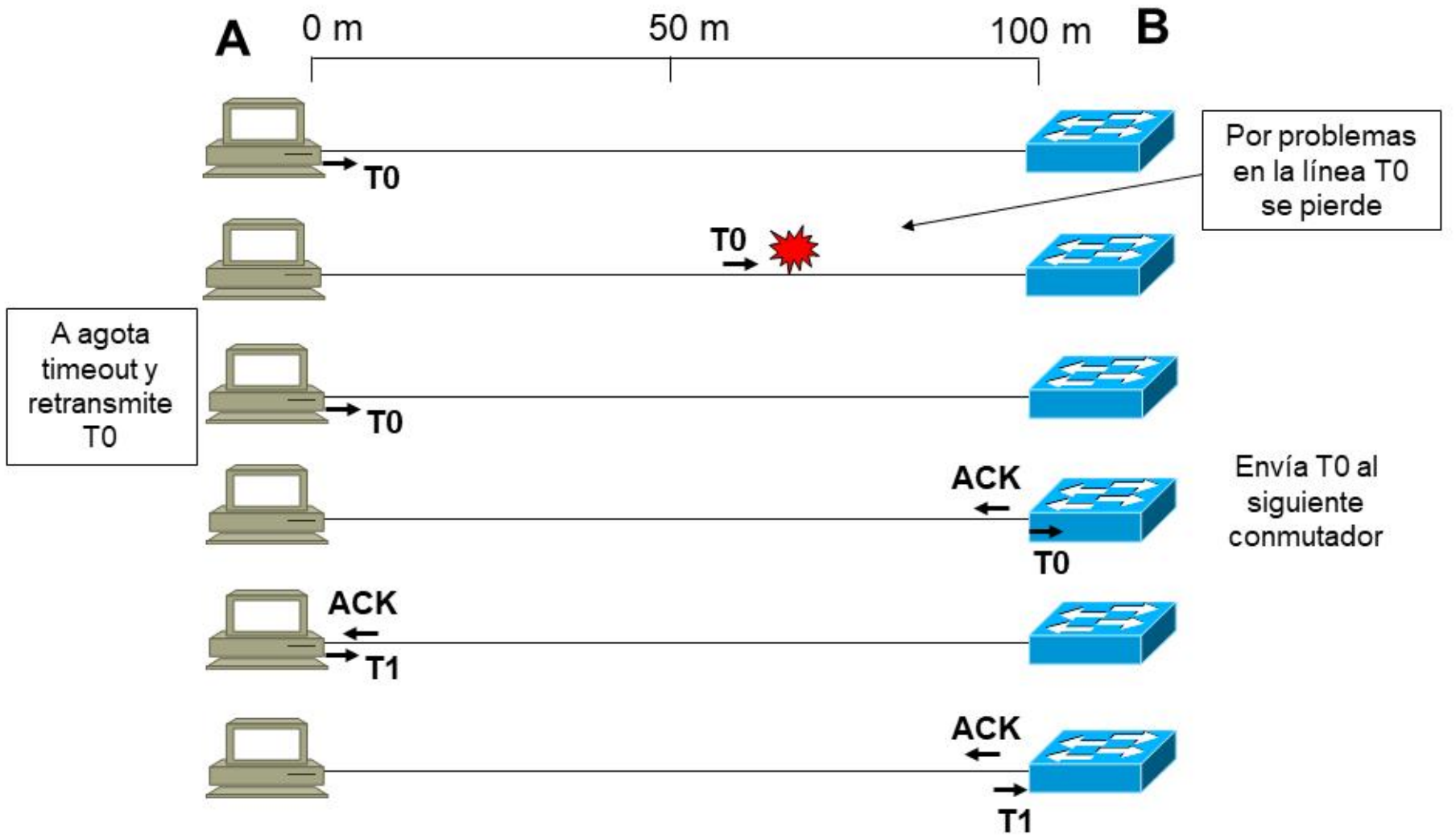
Protocolo con control de flujo



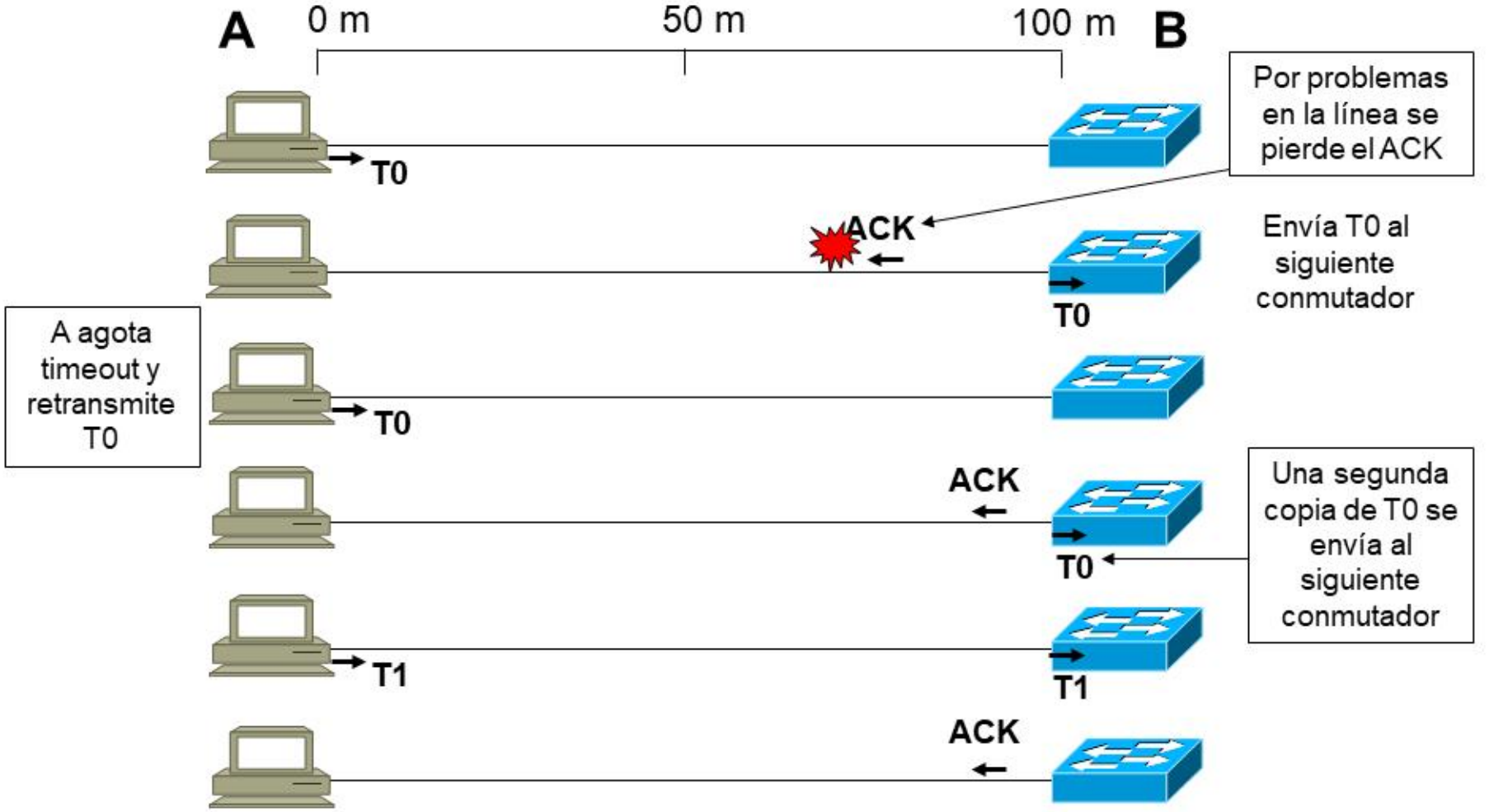
Protocolo resistente a errores I

- Vamos a admitir ahora que pueda haber errores en la transmisión, incluso perderse por completo alguna trama.
- Podemos utilizar el protocolo anterior si convenimos que el ACK indique, no solo espacio disponible en buffers, sino también recepción OK. El emisor reenviará la trama errónea o perdida cuando no reciba el ACK en el plazo de tiempo acordado (timeout).

Protocolo resistente a errores I



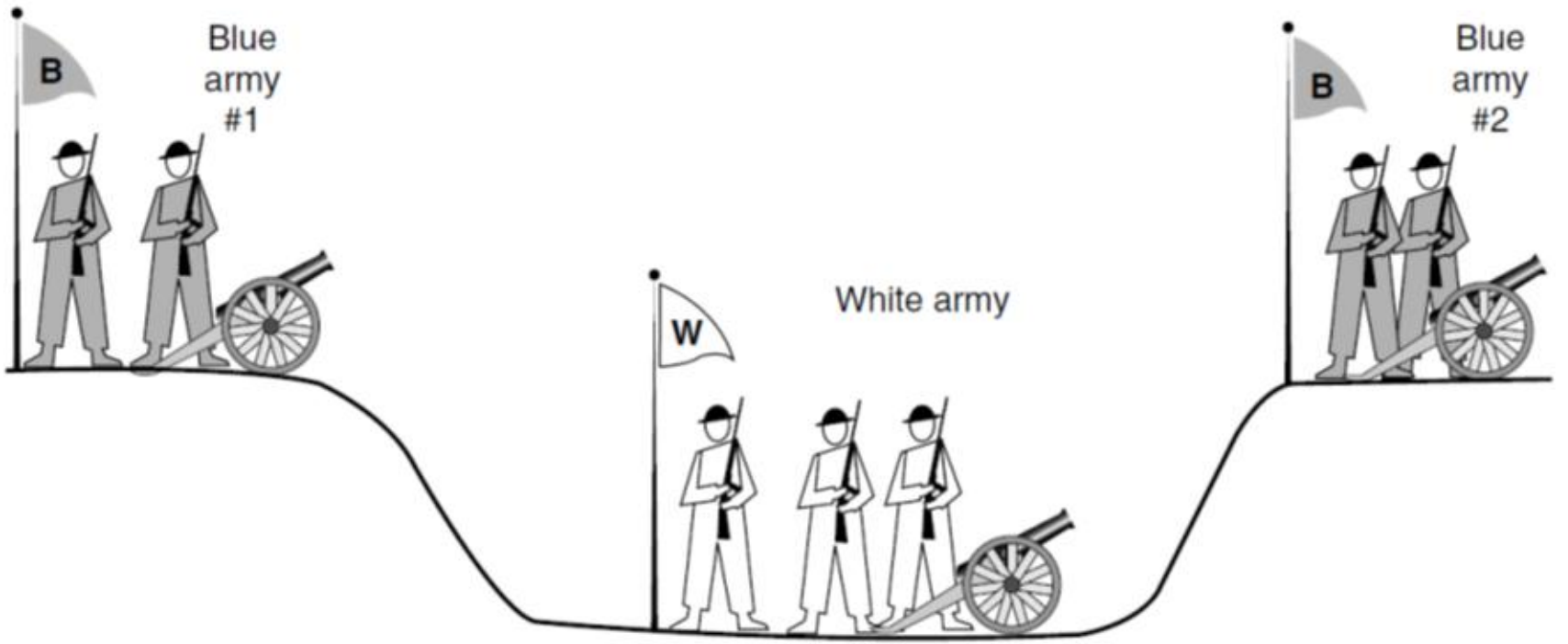
Protocolo resistente a errors I: problema



Posible 'solución' al problema

- Podemos añadir a nuestro protocolo un ACK del ACK, de forma que el receptor sepa cuando se ha perdido su ACK. Pero ¿y si se pierde el ACK del ACK?
- Podemos añadir un ACK del ACK del ACK, y así sucesivamente
- Aparte de la ineficiencia que genera, encadenar ACKs no resuelve el problema. Esto es lo que se conoce como el problema de los dos ejércitos (o de los dos generales)

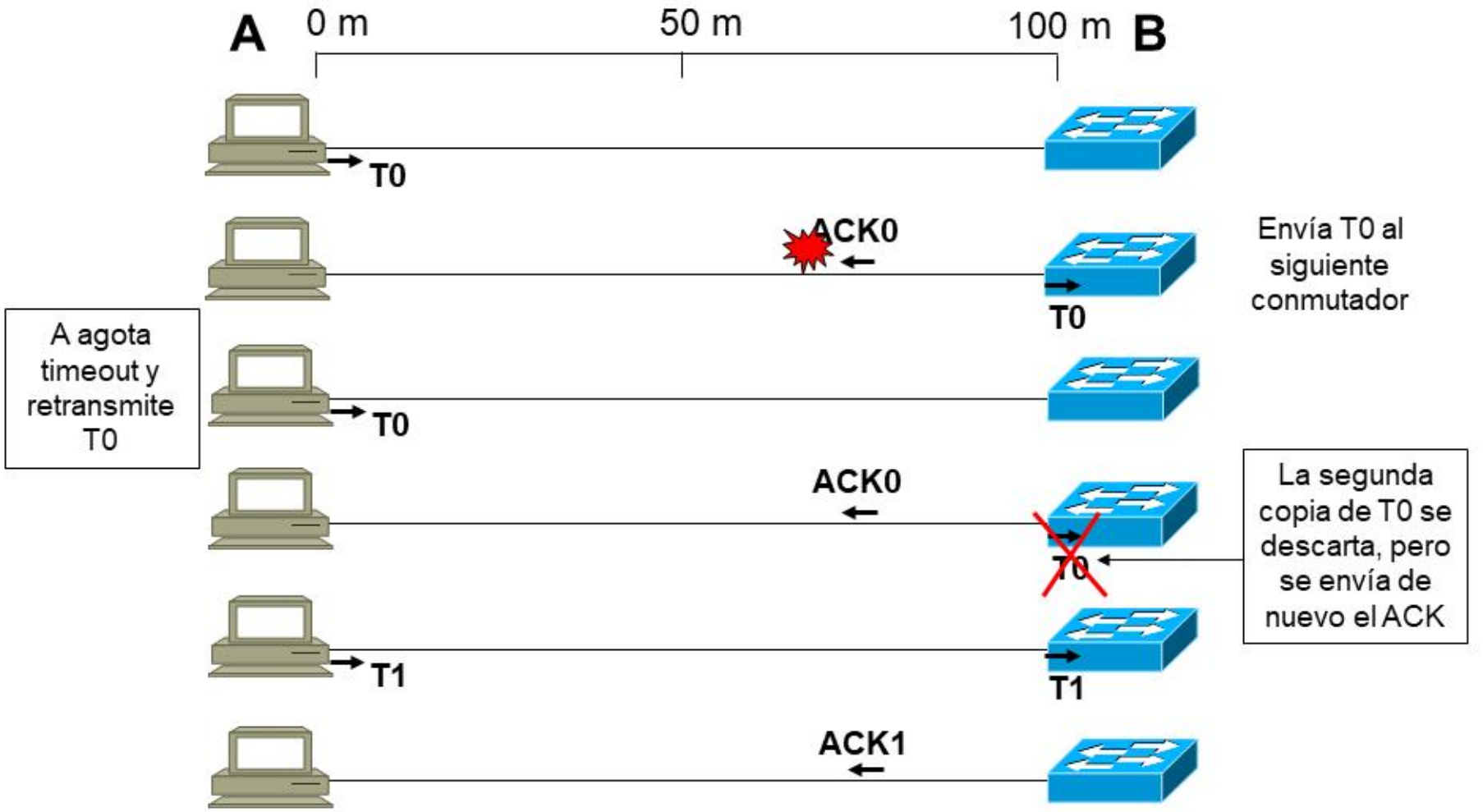
Problema de los dos ejércitos



La verdadera Solución

- El de los dos ejércitos es un problema clásico que no tiene solución, pero el de los ACK sí. Basta con numerar los ACK para asociarlos a las tramas
- En realidad no es preciso un rango de numeración infinito, basta con un bit para numerarlas alternando 0 y 1, ya que solo necesitamos distinguir cada trama (y su ACK) de la que le sigue y la que le precede
- Cuando el receptor reciba una trama duplicada se dará cuenta porque verá que su número coincide con la que acaba de recibir; en ese caso la descartará y repetirá el envío del ACK (de lo contrario el emisor persistirá en el envío indefinidamente)

Protocolo resistente a errores II



Protocolos de parada y espera

- Los ejemplos ejemplos anteriores (protocolo con control de flujo, protocolo resistente a errores I y II) tienen en común la espera del ACK antes de enviar la siguiente trama. Por eso se les denomina 'protocolos de parada y espera'
- En todos ellos el ciclo competo de envío de una trama dura 2,136 μs (un RTT). En este tiempo la línea está ocupada 0,512 μs (t_trans), así que la eficiencia será:

  0,512/2,136 = 0,24 = 24%

Eficiencia en los protocolos de parada y espera

- En general podemos calcular la eficiencia como:

  $$\text{Eficiencia} = t_trans / RTT$$

- Como el ACK suele ser muy pequeño su t_trans se suele considerar despreciable. Si suponemos lo mismo con el tiempo de proceso en destino nos queda que:

  $$RTT = t_trans + 2 * \tau$$

- Así que:

  $$\text{Eficiencia} = t_trans / (t_trans + 2 * \tau)$$

- Si τ es grande en relación con t_trans la eficiencia es muy baja

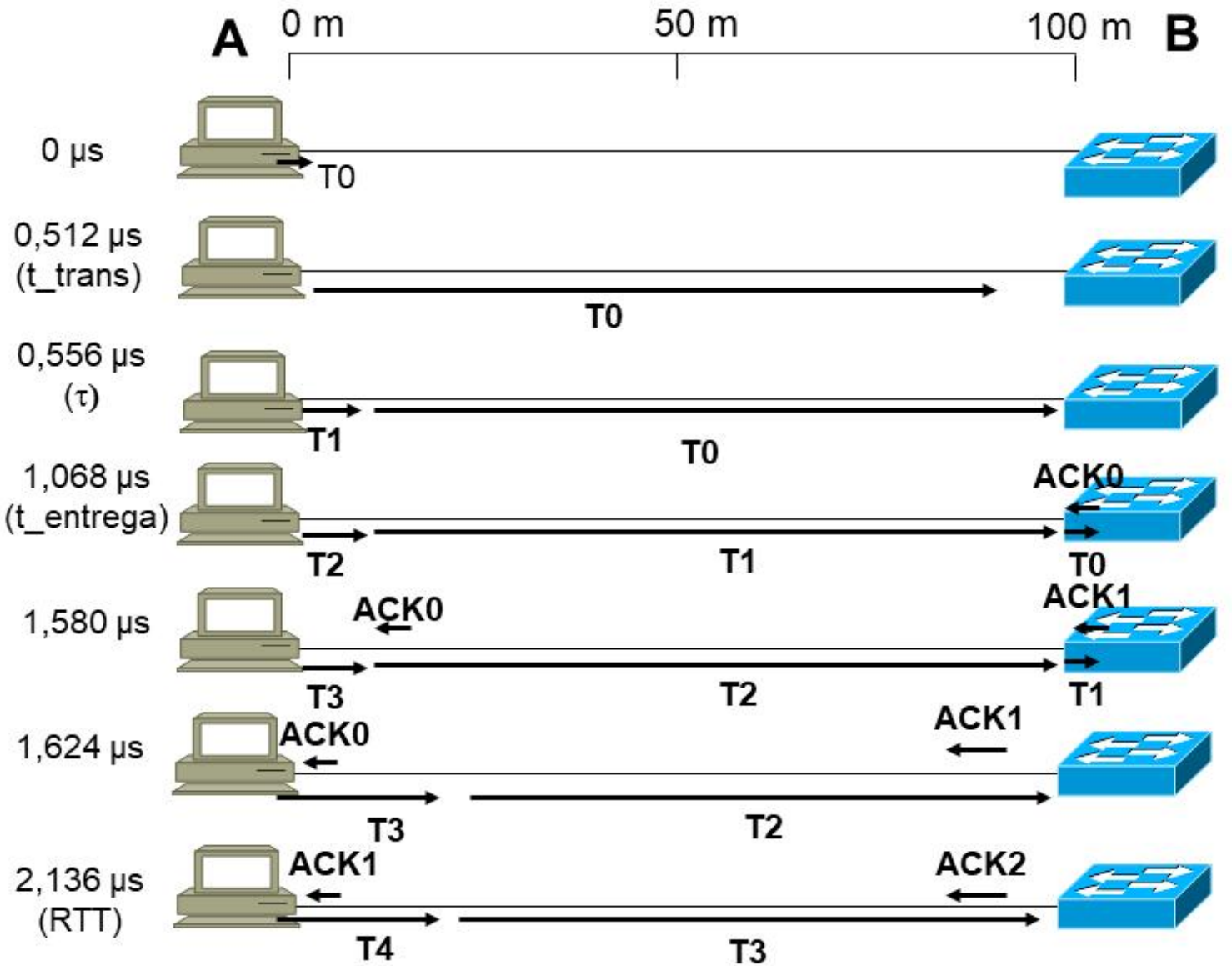
Protocolos de ventana deslizante

- Nuestro protocolo es poco eficiente. Para mejorarlo necesitamos que no pare, y para eso hay que 'llenar' el cable de bits, hacer un 'pipeline'
- Dependiendo de los valores de t_trans y τ el número de tramas necesarias para llenar la tubería puede variar. Solo llenándola podemos conseguir una eficiencia del 100%
- En cada instante podremos tener varias tramas pendientes de confirmación. Es como las tramas fueran una secuencia escrita en una tira de papel tapada y una ventana que se desplaza sobre la tira nos va mostrando las quevan de camino en cada momento

Numeración de las tramas en el protocolo de ventana deslizante

- Como ahora podremos tener varias tramas pendientes de confirmar habrá que numerarlas, y también sus ACKs. Ya no nos valdrá numerar con 0-1, El rango deberá ser mayor que el número máximo de tramas pendientes de confirmar. Así cuando falle alguna sabremos cuál ha sido.
- El número máximo de tramas pendientes de confirmar se conoce como 'tamaño de ventana'

Protocolo de ventana deslizante



Protocolo de ventana deslizante

- Cuando se produce un error hay dos estrategias posibles:
  - **Retroceso-n**: se retransmiten todas las tramas a partir de la errónea
  - **Repetición selectiva**: se retransmite solo la trama errónea y se sigue enviando las tramas como antes del error
- Repetición selectiva es más complejo pero más eficiente. Además requiere más espacio en buffers en el receptor.

Tamaño de ventana

- La ventana mínima para un 100% de ocupación es la que 'llena el hilo' de datos, que se puede calcular con la fórmula:

  W = 2 * τ * caudal / tamaño_trama + 1

  – Ej.: $\tau = 0{,}556\ \mu s$, caudal = 1 Gb/s, tamaño_trama = 512 bits

  $\rightarrow W = 3{,}18 \rightarrow$ Ventana mínima: 4

- Esta fórmula supone:
  – Tiempo de proceso despreciable
  – Longitud de trama ACK cero bytes
  – Mismo τ a la ida que a la vuelta (ruta simétrica)

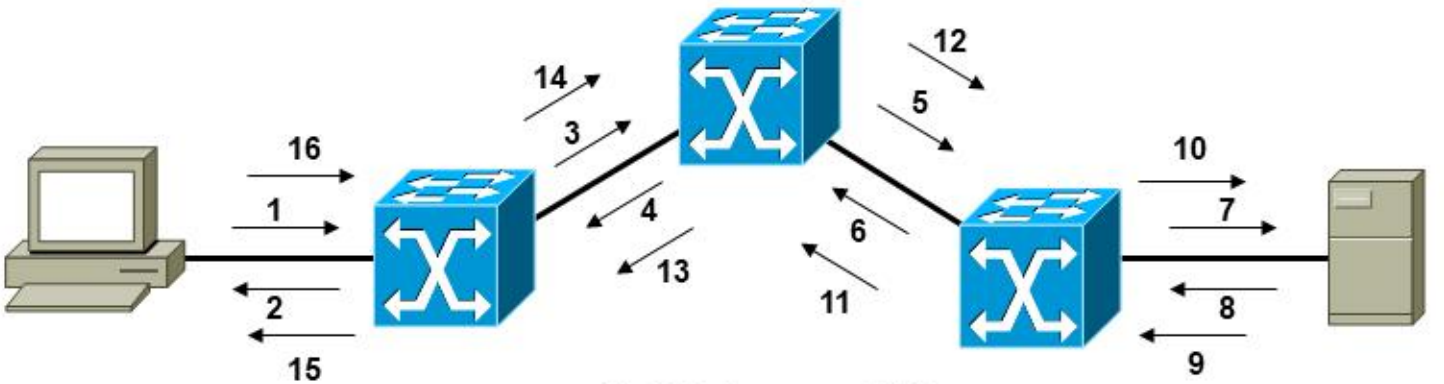
Tamaño de ventana y número de secuencia

- Para que funcione correctamente un protocol de enlace con ventana deslizante se ha de cumplir que la ventana sea como máximo:
  - En retroceso n: *Número de secuencia – 1*
  - En repetición selectiva: *Número de secuencia/2*
- Por ejemplo, si el número de secuencia es un campo de tres bits:
  - En retroceso-n la ventana máxima es de 7
  - En repetición selectiva la ventana máxima es de 4

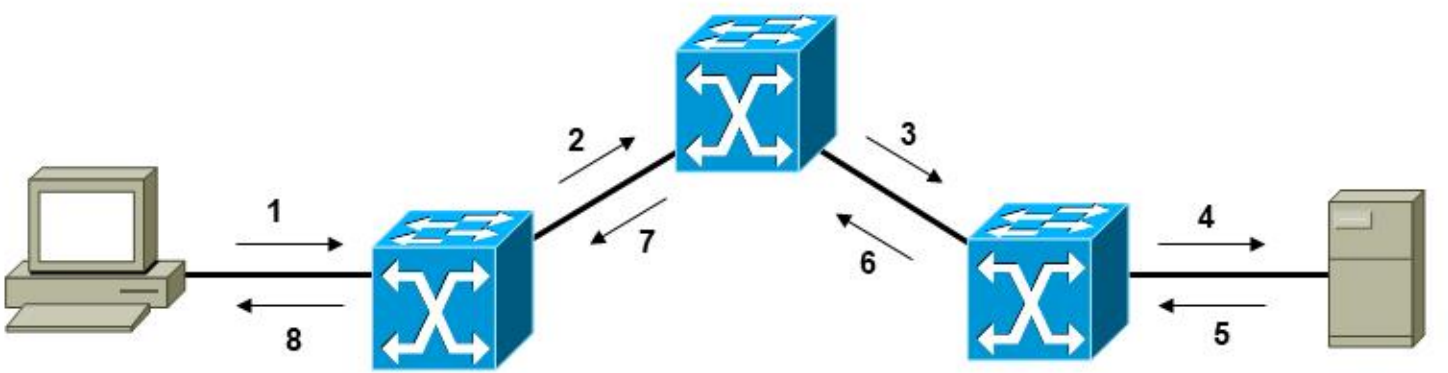
Protocolos de enlace reales

- Antiguamente (años 80) se utilizaban sofisticados protocolos de enlace (ventana deslizante con repetición selectiva) para conseguir el máximo rendimiento en líneas de baja velocidad y BER elevado
- Hoy en día las líneas son de alta velocidad y el BER es muy bajo; en esas situaciones el envío de ACK no sale rentable. La gran mayoría de los protocolos de enlace actuales funciona sin ACK. Se supone que hay muy pocos errores, y no se ejerce control de flujo. Si hay errores la trama se descarta sin más, será el nivel de transporte el que solicite retransmisión en su caso.
- Una excepción notable son las redes inalámbricas, que siguen utilizando ACKs.

Comunicación TCP sobre X.25 y Frame Relay



X.25 (años 80)



Frame Relay (años 90)